



የኢትዮጵያ

ESP32

ዲፕሎማ



iMECHATEUS

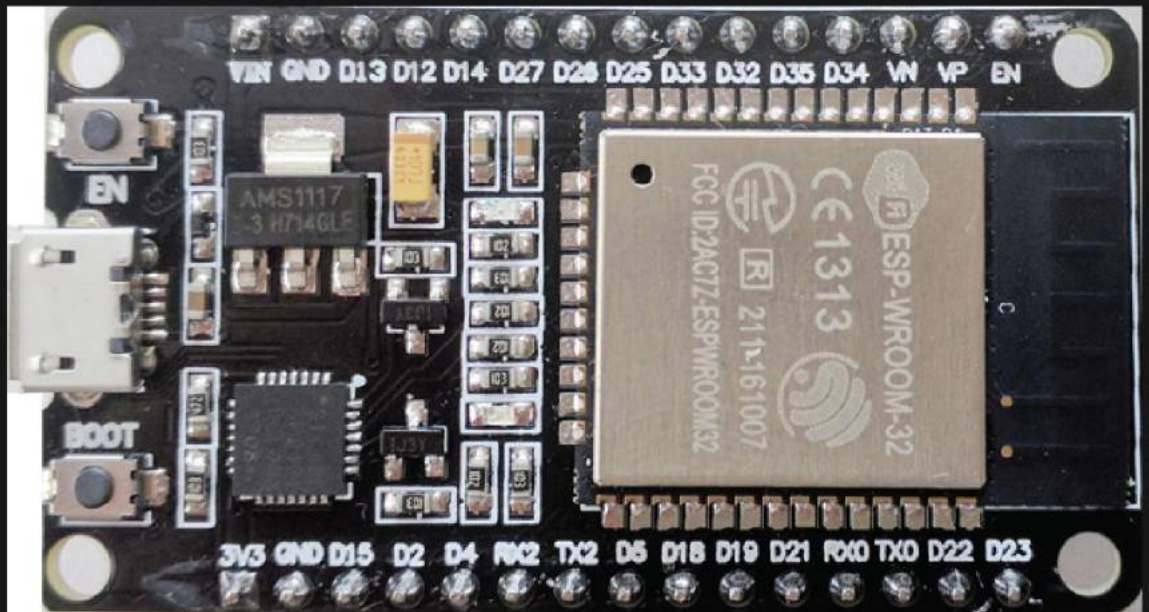
2021

1 - Pengantar ESP32

ESP32 adalah sebuah komputer mini yang digunakan untuk segala macam mesin otomasi yang memiliki fitur canggih namun dengan harga yang sangat terjangkau. Jenis komputer ini sedikit berbeda dengan Personal Computer (PC) ataupun Laptop yang sering kita lihat. ESP32 masuk ke kategori mikrokontroler, yaitu mini komputer yang mengerjakan sistem otomasi tanpa menggunakan spesifikasi yang tinggi seperti pada laptop atau PC.

Mikrokontroler sudah sering kita temui di kehidupan sehari-hari. Dimanapun Anda lihat sistem otomasi, sudah dimungkinkan di dalamnya terdapat mikrokontroler. Contoh sistem-sistem otomasi yang menggunakan mikrokontroler adalah: mesin cuci, vending machine, BLU-RAY player, mesin kasir, dsb.

ESP32 adalah salah satu macam mikrokontroler yang sangat umum untuk digunakan sebagai media pembelajaran. Anda dapat memanipulasi suatu sistem otomasi dengan mikrokontroler dengan kombinasi sensor dan output seperti relay atau motor DC. ESP32 memiliki keunggulan dibandingkan macam mikrokontroler lainnya, yaitu sudah tertanam modul WiFi dan Bluetooth di dalam satu board saja.



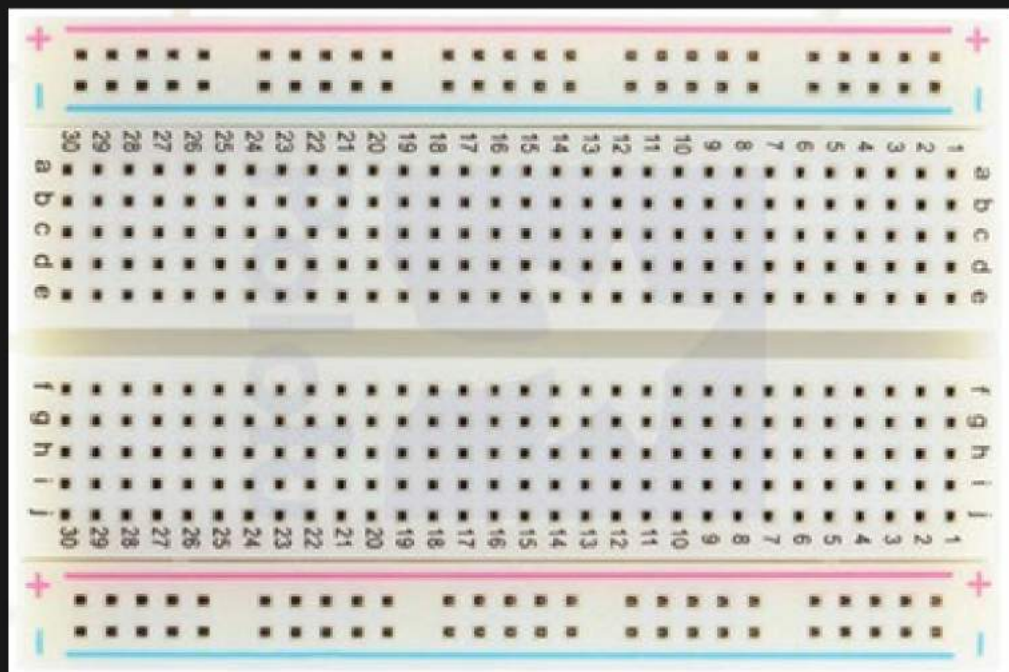
1.1 Perlengkapan Yang Diperlukan

Ada baiknya kita mempelajari mikrokontroler ESP32 dengan perlengkapan yang memadai. Anda akan belajar mengenai rangkaian listrik dasar yang berfungsi untuk menghubungkan sensor-sensor ke ESP32 dan juga perangkat-perangkat output seperti relay dan motor DC. Selain pengetahuan mengenai hardware, Anda juga harus bisa membuat kode untuk menjalankan program mikrokontroler ESP32. ESP32 mengenali bahasa C sebagai bahasa pemrograman

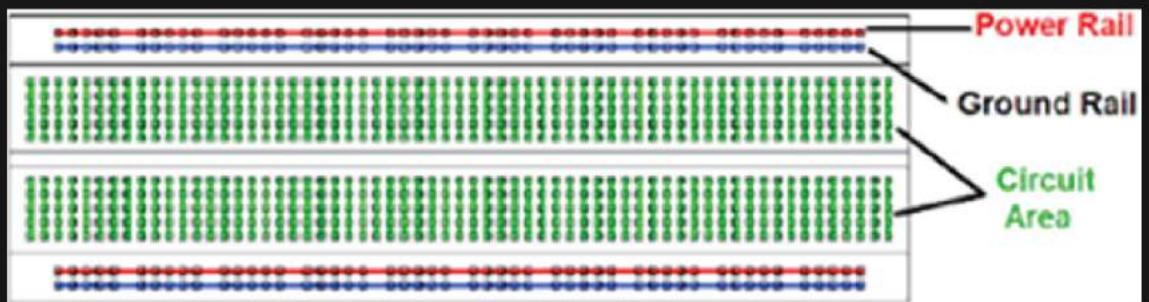
utamanya. Anda akan menggunakan aplikasi Arduino IDE yang sangat mudah digunakan untuk membuat program mikrokontroler ESP32.

Breadboard

Adalah alat yang berfungsi sebagai penghubung jalur-jalur listrik untuk sensor-sensor dan komponen-komponen ke ESP32. Breadboard sering juga disebut dengan project board. Sesuai dengan namanya, bentuknya adalah sebuah papan yang dilengkapi dengan koneksi khusus untuk membantu eksperimen rangkaian listrik.



Ada aturan dalam memasukkan kaki komponen ke dalam breadboard. Jalur-jalur breadboard telah didesain secara ergonomis supaya mempermudah penggunaan dalam rangkaian eksperimen. Berikut bentuk aturan jalurnya.



Dapat Anda lihat garis-garis yang utuh menunjukkan koneksi yang common atau satu jalur. Lazimnya, garis yang berwarna hijau adalah untuk kaki-kaki komponen dan yang berwarna merah-biru digunakan untuk jalur daya dan ground.

Jumper

Merupakan salah satu komponen penting yang berfungsi sebagai kabel penyambung rangkaian. Seringkali Anda akan tidak cukup dengan menancapkan komponen-komponen Anda pada breadboard. Maka untuk membuat rangkaian yang komplit Anda perlu bantuan dari jumper.

Ada 2 macam jumper yang akan sering digunakan yaitu, dupont jumper dan kabel tunggal. Dupont jumper memiliki variasi yaitu Female-Male, Male-Male atau Female-Female. Kabel tunggal tentunya dalam bentuk Male-Male, namun Anda dapat memotongnya sesuai ukuran yang diinginkan.



Komponen-komponen lainnya akan mengikuti sesuai dengan bab yang akan dipelajari nanti. Namun, kemungkinan besar Anda akan memerlukan LED (Light Emitting Diode), Push Button, dan Resistor sebagai komponen pokok di setiap proyek mikrokontroler.

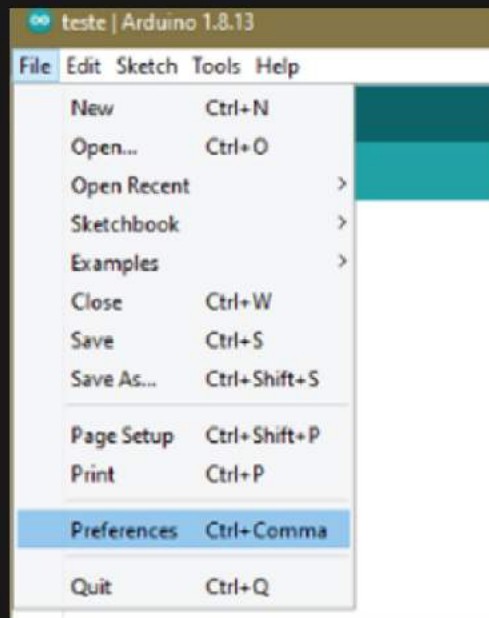
1.2 Arduino IDE

Anda perlu familiarisasi dengan pemrograman pada Arduino IDE. Anda akan belajar bahasa pemrograman secara prosedural yang artinya, perintah yang Anda tulis akan dilaksanakan baris per baris. Ini merupakan filosofi dari bahasa pemrograman C yang menjadi dasar dari Arduino IDE. Instalasi program ini sangatlah mudah. Anda tinggal mengunduh file aplikasinya dari <https://www.arduino.cc/en/software> dan sudah, begitu saja.

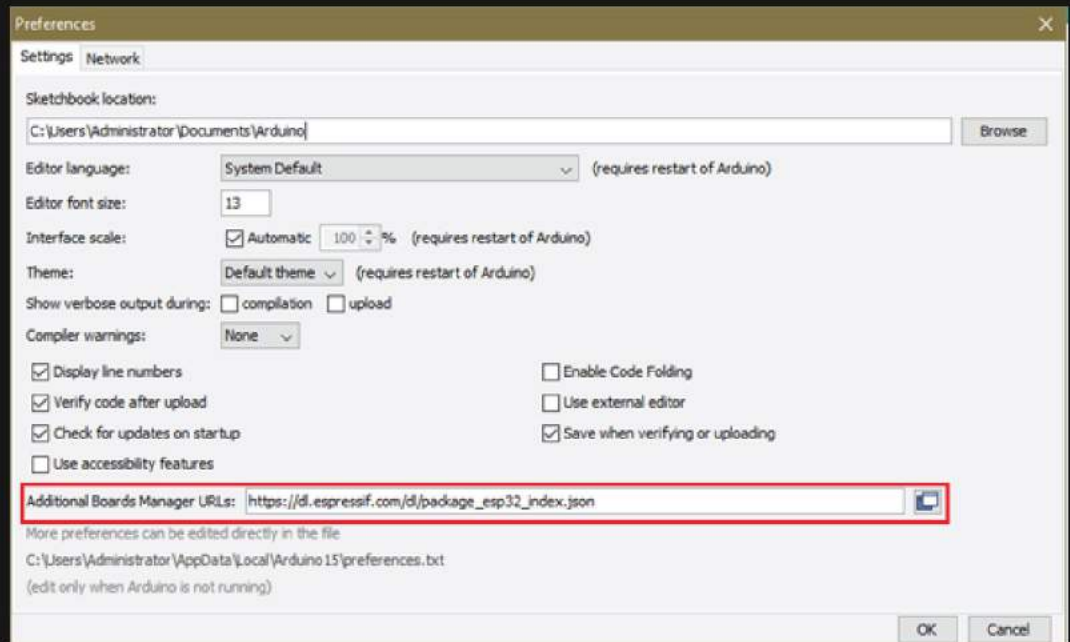
Yang perlu diperhatikan adalah, ESP32 membutuhkan library tambahan untuk dapat diprogram dengan Arduino IDE karena sebenarnya program ini dibuat untuk mikrokontroler keluarga Arduino. Namun, karena fleksibilitas yang tinggi, ESP32 dapat diprogram di sini.

Library ESP32

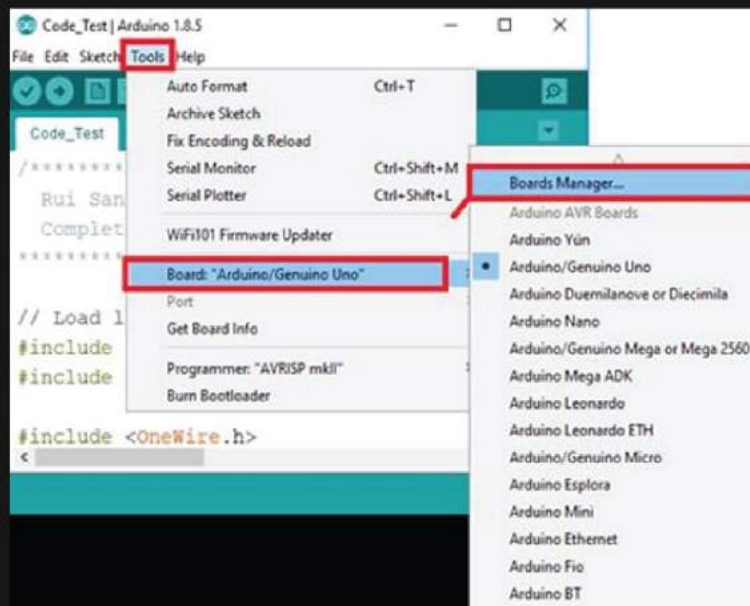
1. Klik File > Preferences.



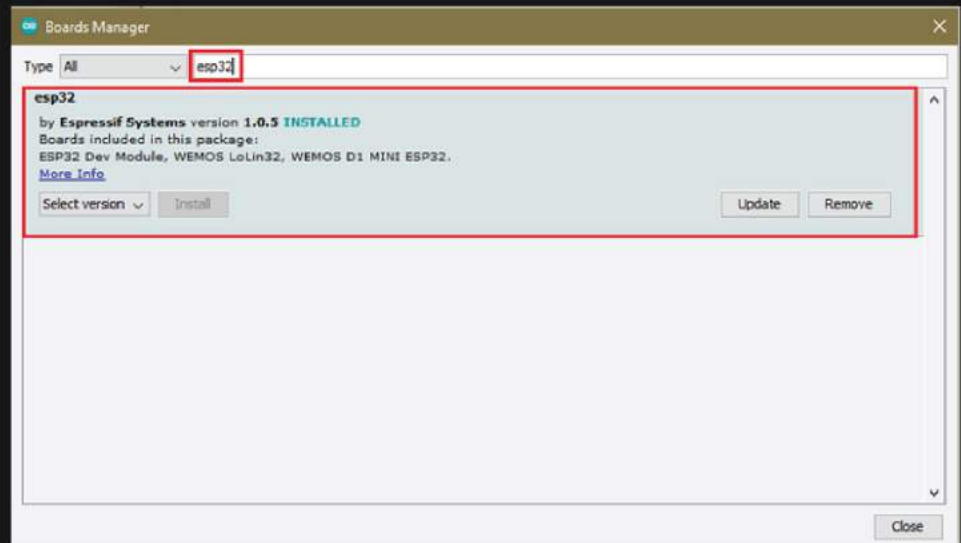
2. Di dalam Anda perlu mengisi sebuah link pada Additional Boards Manager URLs: dengan link https://dl.espressif.com/dl/package_esp32_index.json. Tekan OK.



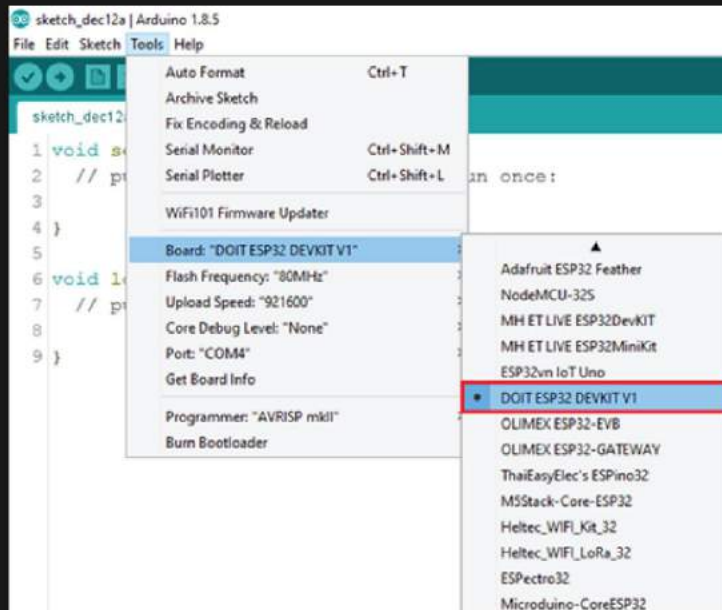
3. Buka Tools > Board > Boards Manager.



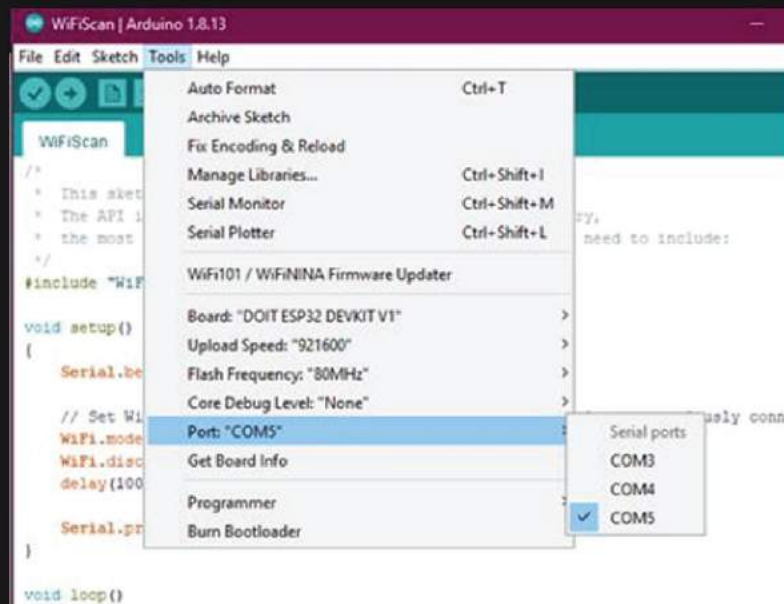
4. Akan muncul window baru untuk mencari esp32. Ketikkan esp32 pada kolom search.



5. Umumnya Anda perlu instal USB Driver CP210x untuk kompatibilitas USB dari DOIT DEVKIT. Link unduh: <https://www.silabs.com/products/development-tools/software/usb-to-uart-bridge-vcp-drivers>. Lalu install seperti program biasa.
6. Ganti board Arduino IDE ke ESP32. Klik Tools > Board, kemudian pilih DOIT DEVKIT V1.



7. Pilih port yang digunakan untuk ESP32. Klik Tools > Port: pilih port yang sesuai.



2 - Pemrograman Dasar ESP32

Kita akan belajar kode-kode program dasar yang sering digunakan pada Arduino IDE. Pertama-tama kita harus mengenal pin dan port yang ada pada ESP32.



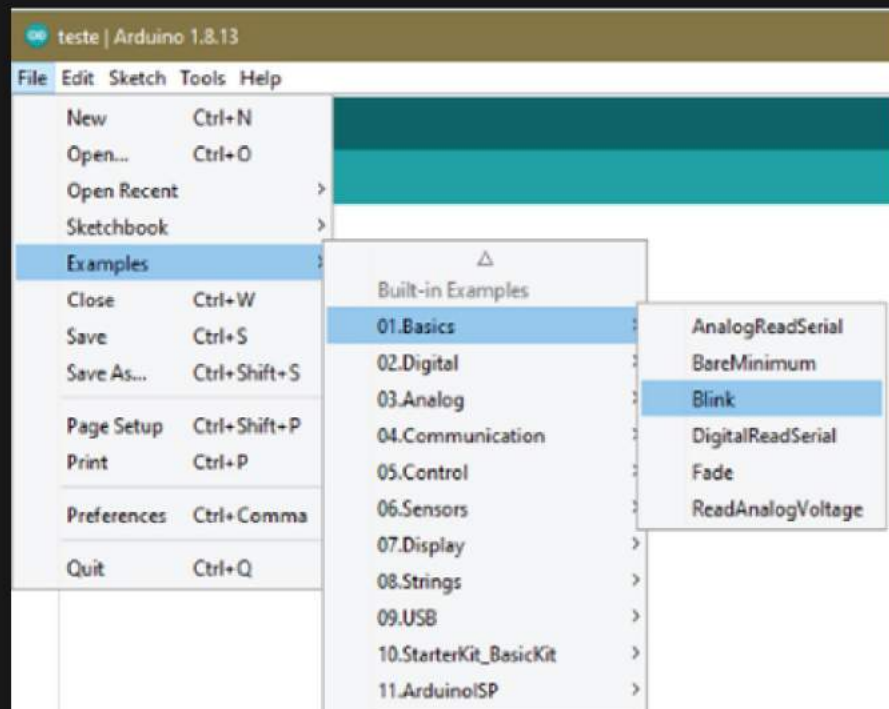
Standar dari DOIT DEVKIT terdapat 30 GPIO yang dapat digunakan. GPIO adalah interface yang digunakan untuk memberi input atau mendapatkan output kepada/dari ESP32. Yang akan dimanipulasi pada mikrokontroler adalah pin GPIO ini untuk mendapatkan sistem otomatisasi yang diinginkan. Contoh: Anda dapat membuat alarm kebakaran online dengan input sensor gas dan output pesan yang dikirim WiFi ke smartphone

Anda.

2.1 Blinking LED

Sebelum masuk ke penulisan kode program, kita pelajari kode yang tersedia pada Arduino IDE examples. Pada examples sudah disiapkan program-program sederhana untuk menjadi wadah pembelajaran seperti blink, WiFi scan, interrupt, dsb. Kali ini kita coba blinking LED yang cukup simpel.

1. Klik File > Examples > Basic > Blink.



2. Anda akan mendapatkan kode seperti di bawah ini.

```
// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}
// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH); // turn the LED on
  delay(1000);                      // wait for a second
  digitalWrite(LED_BUILTIN, LOW);  // turn the LED off
  delay(1000);                      // wait for a second
}
```

3. Unggah program blink ke board ESP32 Anda dengan mengklik Upload.



4. Hasil dari program akan menghasilkan kedipan LED yang tertanam pada ESP32.

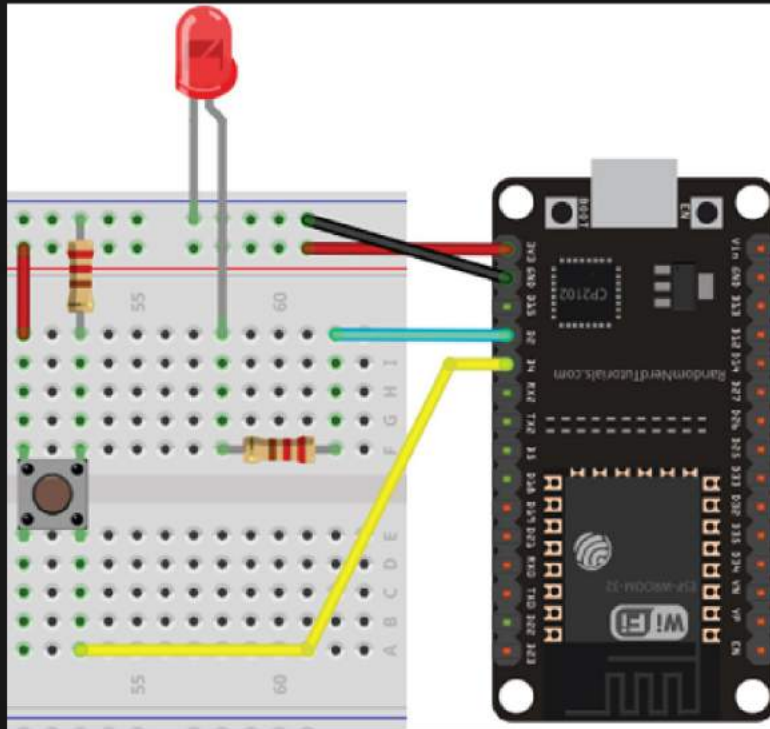


2.2 Digital Read & Write

ESP32 dapat memanipulasi sinyal digital untuk input maupun output. Tegangan maksimum yang dioperasikan ESP32 adalah 3,3 volt dengan ground 0 volt. Dua level tegangan ini digunakan sebagai sinyal digital yaitu logika HIGH pada 3,3V dan LOW pada 0V. Contoh program Digital Read-Write adalah LED yang hidup ketika switch on. Berikut kodenya.

```
void setup() {  
    // initialize the LED pin as an output:  
    pinMode(2, OUTPUT);  
    // initialize the pushbutton pin as an input:  
    pinMode(4, INPUT);  
}  
  
void loop() {  
    // read the state of the pushbutton value:  
    int buttonState = digitalRead(4);  
  
    // check if the pushbutton is pressed. If it is, the buttonState is  
    HIGH:  
    if (buttonState == HIGH) {  
        // turn LED on:  
        digitalWrite(2, HIGH);  
    } else {  
        // turn LED off:  
        digitalWrite(4, LOW);  
    }  
}
```

Wiring diagram menyerupai gambar berikut.



Kode program ini akan menghasilkan output LED yang hidup selama push button ditekan. Ketika push button dilepas maka LED akan mati.

Bila Anda perhatikan ada baris kode yang menggunakan variabel dengan tipe data integer (`int button-State`). Variabel ini digunakan sebagai penyimpan nilai digital 0 atau 1 dari `digitalRead()` yang kemudian dilewatkan pada control flow `if`.

void setup()

Merupakan program utama Arduino IDE sketch untuk memerintahkan mikrokontroler pertama kali dan sekali saja. Umumnya digunakan untuk menentukan kondisi inisial dari seluruh pin yang akan digunakan.

void loop()

Merupakan program utama Arduino IDE sketch untuk melakukan perintah setelah insialisasi void setup() dan terus mengulang baris kode selama ESP32 tidak dimatikan atau reset.

digitalRead(#GPIO)

Memiliki 1 argumen, yaitu pin GPIO beberapa. Fungsi dari digitalRead() adalah untuk membaca sinyal digital yang diterima apakah HIGH atau LOW, dapat juga berupa nilai integer 1 atau 0.

digitalWrite(#GPIO, state)

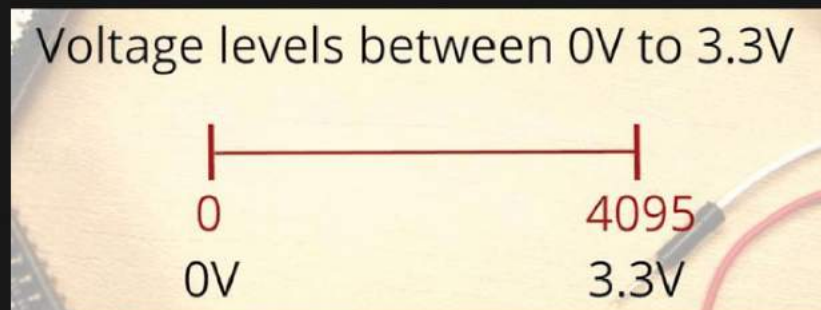
Memiliki 2 argumen, yaitu pin GPIO beberapa dan state sinyal digital apakah HIGH atau LOW. Tujuan dari digitalWrite adalah untuk menghasilkan sinyal digital HIGH atau LOW pada pin ESP32 yang dipilih.

pinMode(#GPIO, mode)

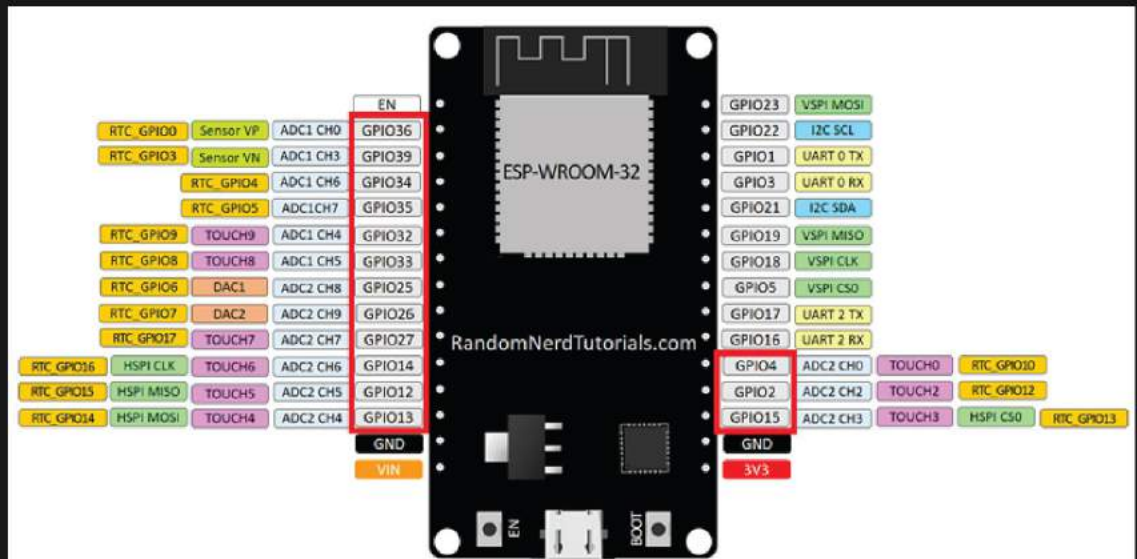
Memiliki 2 argumen, yaitu pin GPIO seberapa dan mode pin tersebut apakah sebagai input atau output.

2.3 Analog Input

Selain sinyal digital, kita juga dapat membaca sinyal analog pada pin input analog ESP32. Pembacaan nilai analog tersebut tergantung dari level tegangan yang di inputkan ke pin analog. ESP32 dapat membedakan level tegangan dari 0V – 3,3V. Nilai tegangan ini dikonversikan ke nilai interger dengan resolusi 12 bit, yang artinya dalam skala 0 – 4095.



Pin-pin yang dapat membaca input analog hanya yang bertanda merah pada gambar di bawah ini. Pin tersebut adalah GPIO: 2, 4, 12, 13, 14, 15, 25, 26, 27, 32, 33, 34, 35, 36, dan 39.



Di bawah ini adalah contoh program analog input yang akan membunyikan piezo buzzer ketika melewati nilai tegangan tertentu.

```
// deklarasi variabel global
const int buzzer_ = 26;
const int potentiometer_ = 27;

// inisialisasi pin
void setup() {
  pinMode(buzzer_, OUTPUT);
}
```

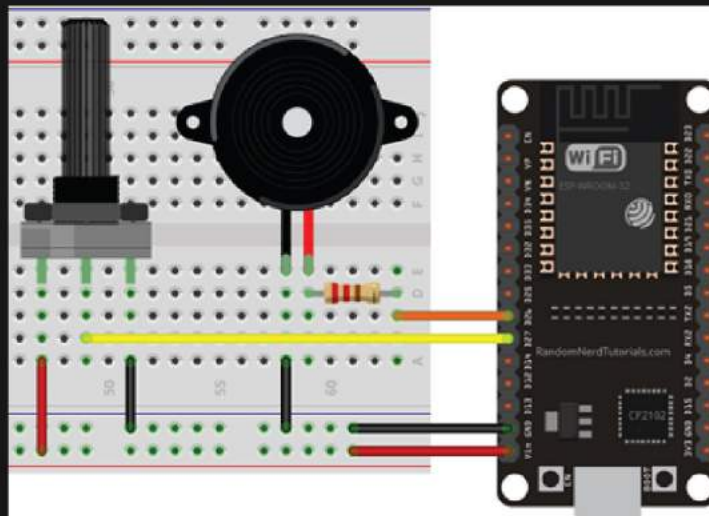
```

void loop() {
  // pembacaan tegangan
  int threshold_ = analogRead(potentio_);
  // syarat buzzer hidup
  if (threshold_ > 3000) {
    digitalWrite(buzzer_, HIGH);
    delay(3000);
    digitalWrite(buzzer_, LOW);
  }
  else {
    digitalWrite(buzzer_, LOW);
  }
}

```

Program ini akan menghasilkan bunyi buzzer apabila pembacaan nilai analog input lebih dari integer 3000. Apabila analog input kurang dari 3000 maka buzzer tidak berbunyi.

Berikut adalah skema rangkaian untuk eksperimen analog input dengan output buzzer.



Serial Monitor

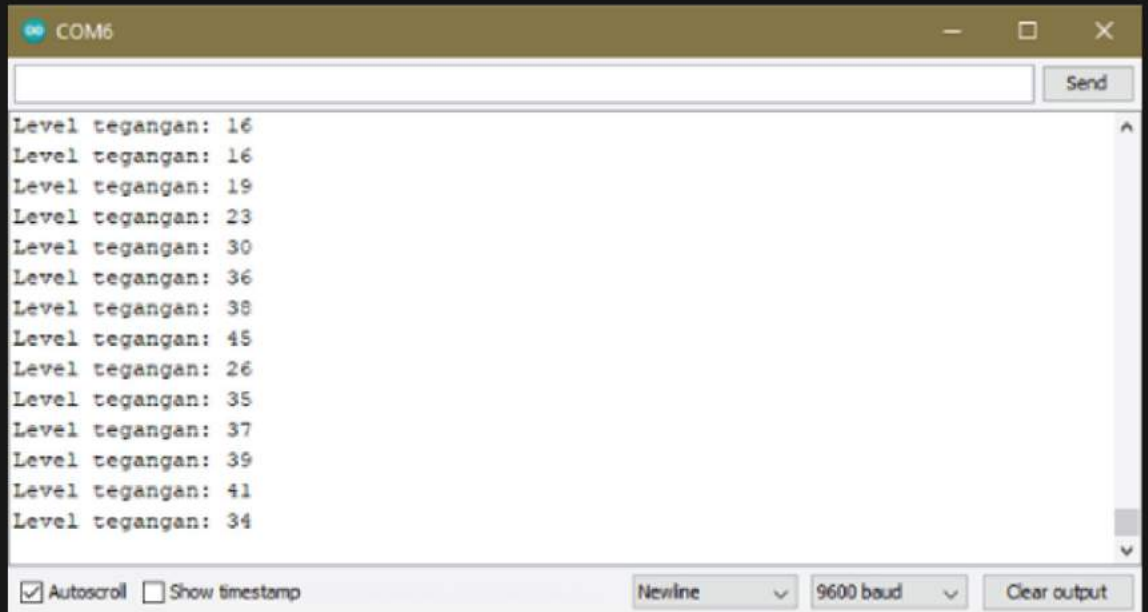
Arduino IDE dilengkapi dengan fitur monitoring yang dapat menampilkan segala output ataupun input yang masuk ke mikrokontroler ESP32. Berikut adalah contoh kode dari serial monitoring.

```
// deklarasi variabel global
const int potentio_ = 27;

// inisialisasi pin
void setup(){
  Serial.begin(9600);
}

void loop(){
  // pembacaan tegangan
  int threshold_ = analogRead(potentio_);
  // menampilkan nilai pada serial monitor
  Serial.print("Level tegangan: ");
  Serial.println(threshold_);
  delay(500);
}
```

Berikut adalah hasil serial monitor yang menunjukkan nilai tegangan yang masuk pada pin input analog.



3 - Pemrograman Lanjut ESP32

Setelah mempelajari pemrograman dasar dari ESP32, kita dapat memasuki pemrograman lanjut yang sedikit lebih kompleks. Di sini Anda akan belajar lebih dalam tentang flow control dari bahasa C dan penggunaan library sebuah sensor.

3.1 Servo

Servo adalah sebuah alat output yang dapat digunakan bersama ESP32. Servo berfungsi sebagai aktuator yang bergerak berotasi dari 0 – 180 derajat. Umumnya, servo digunakan dalam aplikasi penggerak pintu gerbang atau lengan robot.



Servo memiliki 3 pin yaitu jalur data, tegangan supply dan ground. Cara kerja servo adalah bergerak berotasi dengan timing yang berbeda-beda. Semakin cepat waktu maka semakin besar derajat rotasi yang ditempuh dan juga sebaliknya.

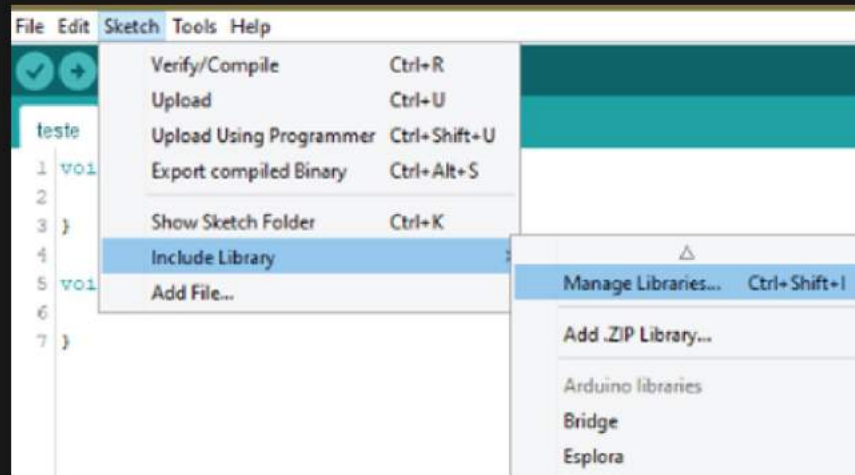
Pengaturan kecepatan putar servo diatur oleh pin jalur data yang menerima kiriman sinyal PWM dari ESP32. Sinyal PWM atau Pulse Width Modulation adalah sinyal DC yang diputus-putus dengan tujuan mengubah daya rata-rata nya. Dengan adanya sinyal PWM Anda dapat menggerakkan servo sesuai lokasi derajat yang Anda inginkan.

Library

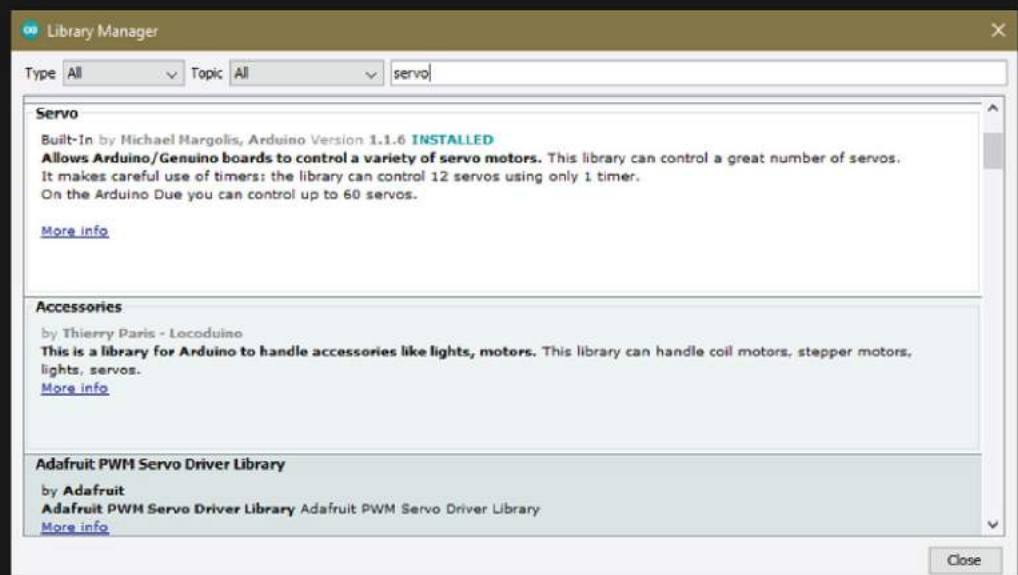
Dengan Arduino IDE, Anda dapat menggunakan bermacam-macam library yang ada di internet. Kebanyakan sensor yang akan Anda gunakan membutuhkan library tambahan supaya pemrograman menjadi lebih mudah. Kali ini akan dicoba memasukkan library Servo ke dalam Arduino IDE.

Berikut langkah-langkah yang perlu dilakukan.

1. Klik Sketch > Include Library > Manage Libraries.



2. Pada search bar, tuliskan servo. Kemudian klik install.



Setelah ter-install maka Anda dapat memulai menulis program untuk servo. Berikut contoh programnya.

```
#include <Servo.h>

Servo test_servo;
const int button_first = 2;
const int button_second = 4;

void setup() {
  test_servo.attach(15);
  pinMode(button_first, INPUT_PULLUP);
  pinMode(button_second, INPUT_PULLUP);
}

void loop() {
  int value_1 = !digitalRead(button_first);
  int value_2 = !digitalRead(button_second);
  delay(100);

  if (value_1 && !value_2) {
    for (int i=0; i<=180; i++) {
      test_servo.write(i);
      delay(30);
    }
  }

  if (value_2 && !value_1) {
    for (int i=180; i>=0; i--) {
      test_servo.write(i);
      delay(30);
    }
  }
}
```

Pada program ini terdapat 2 buah input yaitu push button dan 1 buah output yaitu servo. Servo akan bergerak dari 0 – 180 derajat apabila push button pertama ditekan. Sebaliknya, servo akan bergerak dari 180 – 0 derajat apabila push button kedua ditekan. Ada beberapa hal baru yang Anda lihat pada kode tersebut.

#include <library.h>

`#include` digunakan untuk menyisipkan sebuah library ke dalam program. Library berfungsi untuk meringkas baris kode sehingga penulis program hanya perlu memasukkan fungsi-fungsi yang sudah disiapkan. Contoh di atas menunjukkan penggunaan library servo, ditulis `<Servo.h>`. Anda tinggal perlu memasukkan GPIO yang ingin dipakai pada `nama_servo.attach()` dan untuk menggerakkannya ditulis `nama_servo.write()`.

Tentunya, Anda dapat memprogram servo tersebut tanpa menggunakan library, namun sangat tidak disarankan, karena penulisan kode program yang akan lebih panjang. Jika Anda ingin mencoba, servo memerlukan PWM untuk bekerja, sehingga Anda harus memprogram dengan PWM.

Object

Merupakan syntax untuk membuat obyek. Dalam kasus ini Anda memberi variabel untuk penamaan servo yang akan Anda pakai. Pada contoh kode di atas digunakan nama `test_servo`.

4 - Koneksi Wireless ESP32

Salah satu kelebihan ESP32 dengan mikrokontroler lainnya adalah kemampuan untuk koneksi wireless tanpa tambahan modul eksternal. Pada ESP32, sudah tertanam modul WiFi dan Bluetooth yang dikemas dalam ukuran yang sangat minimalis. Pada bab ini akan dibahas cara memprogram WiFi dan Bluetooth pada ESP32.

4.1 WiFi

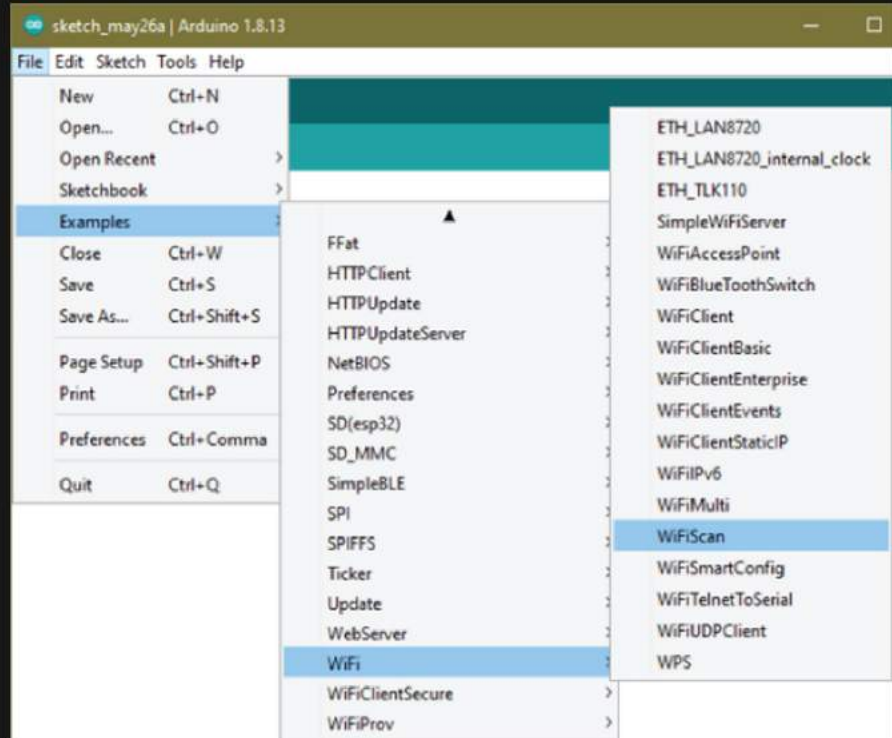
Program paling sederhana untuk testing koneksi WiFi pada ESP32 adalah WiFi scan, yaitu program yang bertujuan untuk menampilkan WiFi Access Point yang terdeteksi di sekitar ESP32 yang Anda gunakan.



Berikut adalah blok diagram dari WiFi scan ESP32. Pada Serial Monitor akan dapat dilihat daftar Access Point yang terdeteksi oleh ESP32.

Pada Arduino IDE, Anda dapat menggunakan Example Program WiFi scan. Berikut lokasi Example-nya.

1. Klik File > Examples > WiFi > WiFiScan.



2. Kode tersebut terlihat seperti berikut.

```
/*  
 * This sketch demonstrates how to scan WiFi networks.  
 * The API is almost the same as with the WiFi Shield library,  
 * the most obvious difference being the different file you need to  
include:  
 */  
#include "WiFi.h"
```

```

void setup()
{
    Serial.begin(115200);

    // Set WiFi to station mode and disconnect from an AP if it was
    previously connected
    WiFi.mode(WIFI_STA);
    WiFi.disconnect();
    delay(100);

    Serial.println("Setup done");
}

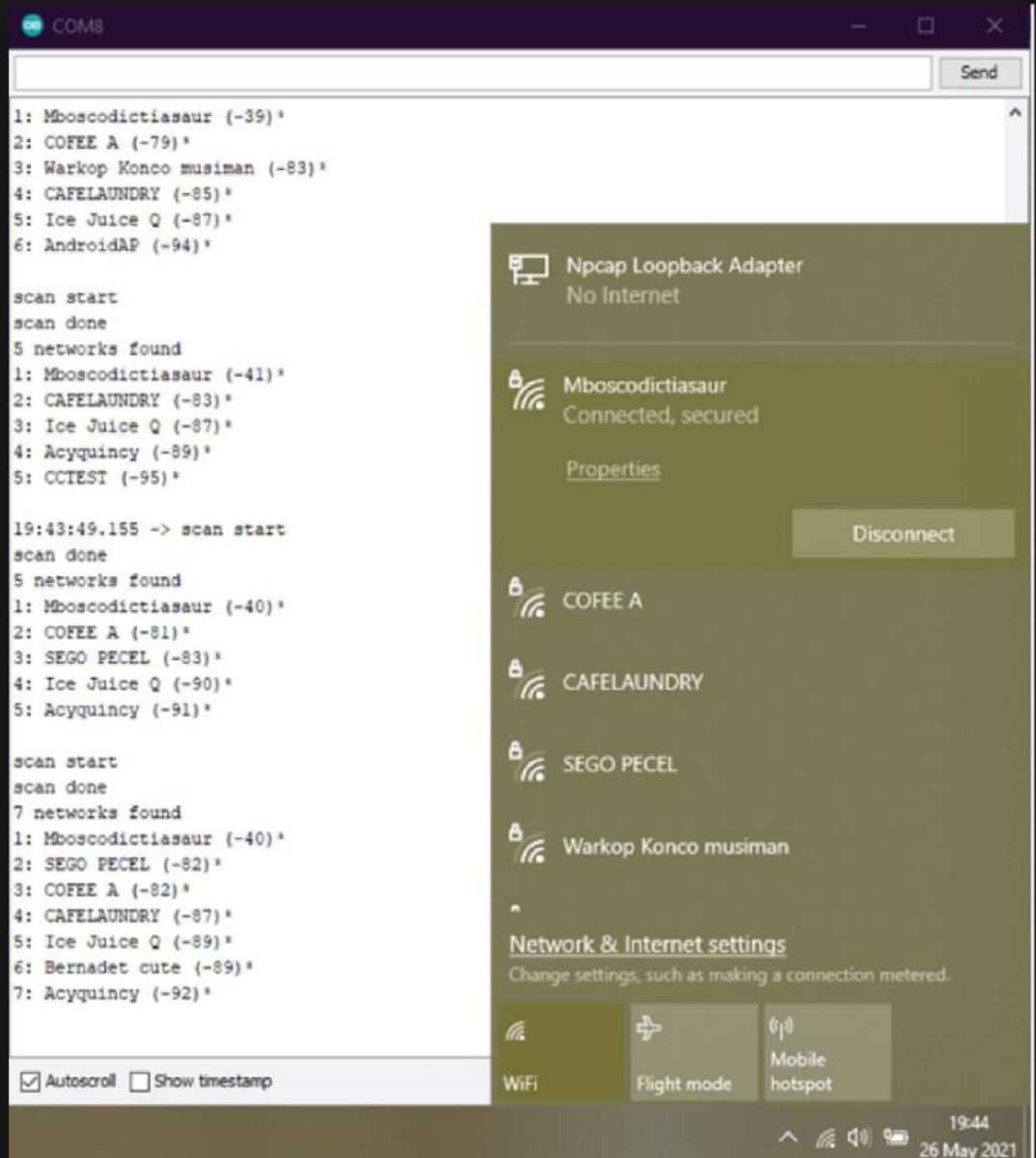
void loop()
{
    Serial.println("scan start");

    // WiFi.scanNetworks will return the number of networks found
    int n = WiFi.scanNetworks();
    Serial.println("scan done");
    if (n == 0) {
        Serial.println("no networks found");
    } else {
        Serial.print(n);
        Serial.println(" networks found");
        for (int i = 0; i < n; ++i) {
            // Print SSID and RSSI for each network found
            Serial.print(i + 1);
            Serial.print(": ");
            Serial.print(WiFi.SSID(i));
            Serial.print(" (");
            Serial.print(WiFi.RSSI(i));
            Serial.print(")");
            Serial.println((WiFi.encryptionType(i) ==
WIFI_AUTH_OPEN)? " W":"");
            delay(10);
        }
        Serial.println("");

        // Wait a bit before scanning again
        delay(5000);
    }
}

```

3. Setelah Anda Upload kode tersebut ke ESP, maka akan muncul hasil scan pada Serial Monitor Anda.



Pada hasil capture di atas dapat Anda lihat hasil scan pada Serial Monitor dan notifikasi WiFi pada Windows 10. Selain nama dari Access Point, program WiF-iScan juga menampilkan kekuatan sinyal dalam satuan desibel. Semakin kecil nilai desibel maka sinyal semakin buruk (semakin negatif).

Webserver

Selanjutnya kita akan mencoba membuat jaringan sederhana Client – Server. Istilah ini dapat diibaratkan seperti pengunjung dan perpustakaan. Pengunjung merupakan Client yang ingin meminjam informasi. Pihak perpustakaan adalah penyedia informasi yang bertindak sebagai Server.

Pada jaringan internet, Client dan Server masing-masing memiliki alamat IP yang umumnya Anda ketahui dalam bentuk 4 angka yang dipisahkan titik-titik, contoh : 192.168.100.1.

Anda dapat menggunakan Example Program dari Arduino IDE, untuk mengimplementasikan koneksi Client – Server. Pada program tersebut, ESP32 akan bertindak sebagai server, dimana Smartphone/Laptop Anda bertindak sebagai Client. Ketika Client mengakses Server, maka Server akan merespon dengan sebuah kalimat pada browser Anda dan juga menge-

dipkan lampu LED pada ESP32.

```
#include <WiFi.h>
#include <WiFiClient.h>
#include <WebServer.h>
#include <ESPmDNS.h>

// isi ssid & password sesuai WLAN lokal Anda
const char* ssid = "...";
const char* password = "...";

WebServer server(80);

const int led = LED_BUILTIN;

void handleRoot() {
  digitalWrite(led, 1);
  delay(1000);
  server.send(200, "text/plain", "hello from esp32!");
  digitalWrite(led, 0);
}

void handleNotFound() {
  digitalWrite(led, 1);
  String message = "File Not Found\n\n";
  message += "URI: ";
  message += server.uri();
  message += "\nMethod: ";
  message += (server.method() == HTTP_GET) ? "GET" : "POST";
  message += "\nArguments: ";
  message += server.args();
  message += "\n";
  for (uint8_t i = 0; i < server.args(); i++) {
```

```

        message += " " + server.argName(i) + ": " + server.arg(i) + "\n";
    }
    server.send(404, "text/plain", message);
    digitalWrite(led, 0);
}

void setup(void) {
    pinMode(led, OUTPUT);
    digitalWrite(led, 0);
    Serial.begin(115200);
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);
    Serial.println("");

    // Wait for connection
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("");
    Serial.print("Connected to ");
    Serial.println(ssid);
    Serial.print("IP address: ");
    Serial.println(WiFi.localIP());

    if (MDNS.begin("esp32")) {
        Serial.println("MDNS responder started");
    }

    server.on("/", handleRoot);

    server.on("/inline", []() {
        server.send(200, "text/plain", "this works as well");
    });

    server.onNotFound(handleNotFound);

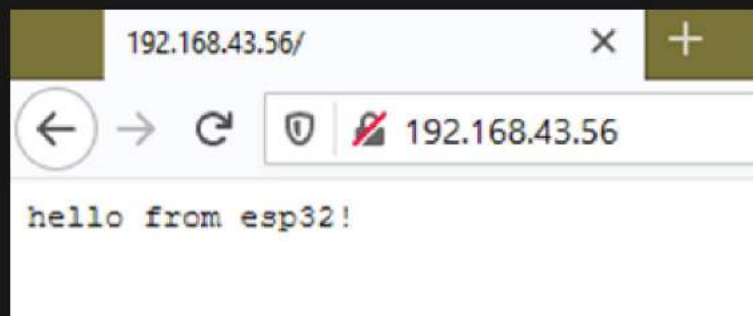
    server.begin();
    Serial.println("HTTP server started");
}

```

```
}  
  
void loop(void) {  
    server.handleClient();  
    delay(2); //allow the cpu to switch to other tasks  
}
```

Setelah Anda jalankan program di atas, buka Serial Monitor untuk mendapatkan alamat IP dari ESP32 Anda (tiap jaringan pasti berbeda).

Kemudian akses browser Anda dari Laptop/Smart-phone dengan memasukkan IP dari ESP32 Anda ke pencarian.



Hasil dari pencarian adalah kalimat "hello from esp32!" yang dapat anda sunting pada sketch Arduino. Selain itu, LED built in pada ESP Anda akan hidup selama 1 detik kemudian padam.

Catatan penting dalam percobaan ini adalah Anda hanya dapat membuat jaringan Client – Server

pada WLAN lokal Anda saja. Untuk akses secara global perlu digunakan teknik khusus dengan tunneling.

4.2 Bluetooth

Perbedaan WiFi dengan Bluetooth adalah tipe jaringan yang digunakan. WiFi tentunya menggunakan internet sebagai jaringan utama untuk menghubungkan ESP dengan Smartphone Anda. Sementara Bluetooth, menggunakan jaringan peer-to-peer, yaitu jaringan dari titik ke titik antar ESP32 dan Smartphone Anda.

Untuk mempermudah pengerjaan, digunakan bantuan aplikasi Serial Bluetooth Terminal yang perlu Anda install pada Smartphone Anda. Link: (https://play.google.com/store/apps/details?id=de.kai_morich.serial_bluetooth_terminal).

```

#include "BluetoothSerial.h"

BluetoothSerial ESP_BT; //Object for Bluetooth

int incoming;
int led_ = LED_BUILTIN;

void setup() {
  Serial.begin(9600); //Start Serial monitor in 9600
  ESP_BT.begin("ESP32_LED_Control"); //Name of your Bluetooth Signal
  Serial.println("Bluetooth Device is Ready to Pair");

  pinMode (led_, OUTPUT); //Specify that LED pin is output
  digitalWrite(led_, 0);
}

void loop() {

  if (ESP_BT.available()) //Check if we receive anything from
Bluetooth
  {
    incoming = ESP_BT.read(); //Read what we receive
    Serial.print("Received:"); Serial.println(incoming);

    if (incoming == 49)
    {
      digitalWrite(led_, HIGH);
      ESP_BT.println("LED turned ON");
    }

    if (incoming == 48)
    {
      digitalWrite(led_, LOW);
      ESP_BT.println("LED turned OFF");
    }
  }
  delay(20);
}

```

Setelah Upload kode di atas ke ESP32 Anda, lakukan pairing Bluetooth antar Smartphone dengan "ESP32_LED_Control". Kemudian buka aplikasi Serial Bluetooth Terminal pada Android Anda dan koneksikan Smartphone dengan ESP32. Ketikkan 1 pada console untuk menyalakan LED, atau 0 untuk mematikan LED pada ESP32.

